



ExtMem: Enabling Application-Aware Memory Page Placement Policies and Mechanisms



Sepehr Jalalian, **Shaurya Patel**, Milad Rezaei Hajidehi, Margo Seltzer, and Alexandra (Sasha) Fedorova

USENIX ATC'24



Memory is expensive

“In recent years, DRAM has become a critical bottleneck for scaling the WSCs” – Google, 2019

“DRAM device scaling has slowed down and it accounts for over 30% of datacenter cost” – Meta, 2022



首先，在google和Meta的报告中都指出内存很昂贵

Applications are memory hungry

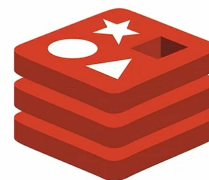
Diverse applications exhibit diverse memory access patterns



Graph Processing



Machine Learning



Databases

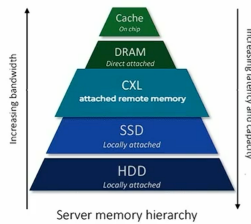


并且，各种类型的应用程序需要大量的内存

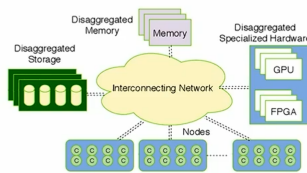
不同程序有不同的内存使用方式（图处理：稀疏且高度随机的指针追逐；ML模型：密集的、流式顺序访问；Database：混合模式）

System setups are heterogenous

Memory Tiering



Disaggregation



Accelerators



Memory management policies need to be tailored to specific applications and hardware

Tiering image source - Rambus



为了减少内存消耗，数据中心的运营商使用了许多技术

内存分层在DRAM内存下加了一层CXL连接的速度较慢但容量更大的拓展内存，通过将频繁访问的热数据迁移到DRAM，将不频繁访问的数据迁移到远程内存，扩大内存容量并降低硬件成本

内存分解将内存资源从计算服务器中物理分离，并通过高速网络将其汇集成一个共享资源池；服务器可以按需从远端的资源池中获取内存（解决单台服务器内存浪费或不足）

另一种方法是使用加速器，用专门的硬件分担CPU在网络和存储I/O等任务上的负载（**Intel VT-d (Virtualization Technology for Directed I/O)** 等技术允许虚拟机直接访问硬件设备（如网卡），绕过hypervisor的软件模拟层，显著降低了I/O延迟）

而内存管理策略需要针对特定的硬件，和在硬件上运行的应用程序进行定制

Overarching Problem

- Current kernel memory management policies are insufficient
- Large overhead of existing userspace upcall (userfaultfd) mechanisms
- Rapidly changing environment there is a need to support new scenarios but development inside kernel is hard

We need a solution that makes it **easy to develop new policies** for different systems **while maintaining performance** equivalent to the kernel!



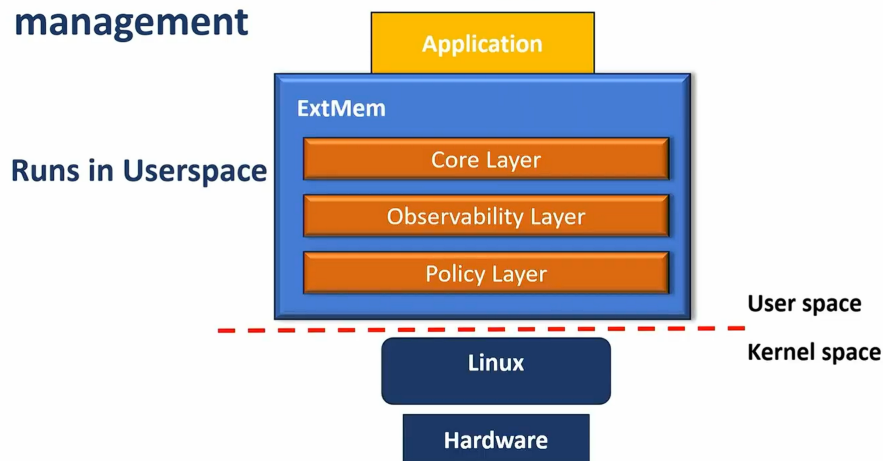
所以，总体的问题是，当前的内存管理策略不足以应对以上说的多种硬件结构、多种应用程序的复杂情况

新的内存管理策略的开发需要涉及到大量的内核代码修改，赶不上环境的快速变化（难以跟上云计算、边缘计算、AI加速器等新兴计算环境的快速演进需求）

有一种叫做userfaultfd的机制，允许将page fault转发给用户态程序处理，以实现**自定义的内存管理策略**，但这样做的开销很大，每次page fault都要面临故障线程与处理线程间的IPC通信成本，在高并发场景下还面临串行化瓶颈问题

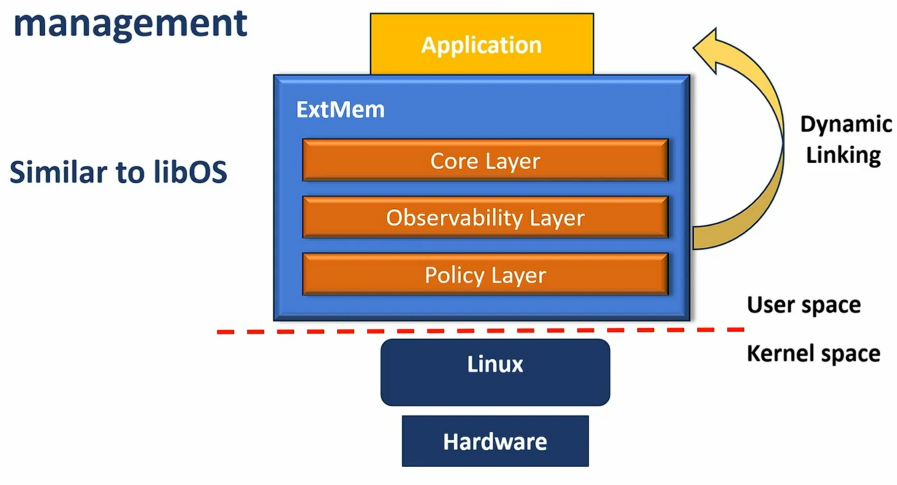
所以，这篇论文提出一个解决方案：它能够定制不同的内存管理策略变得容易，同时，将它的开销维持在和在linux内核中的策略同一水平

ExtMem: a framework for application-specific memory management



这篇论文实现了ExtMem框架，它运行在用户态，允许用户在用户态实现自定义的内存管理策略

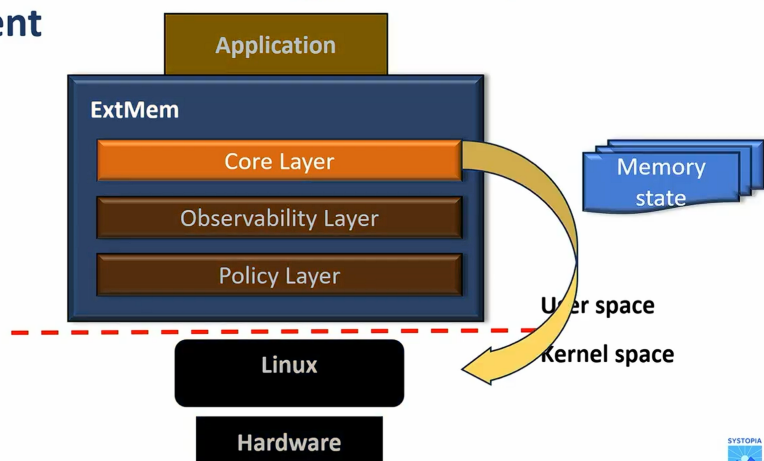
ExtMem: a framework for application-specific memory management



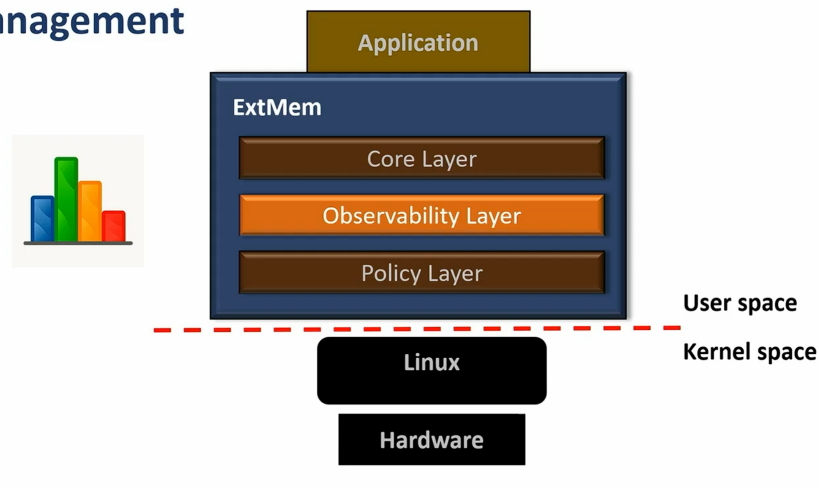
EXTMEM被设计为一个动态链接库，可以被动态地加载到应用程序地址空间中

ExtMem有三个组件

ExtMem: a framework for application-specific memory management

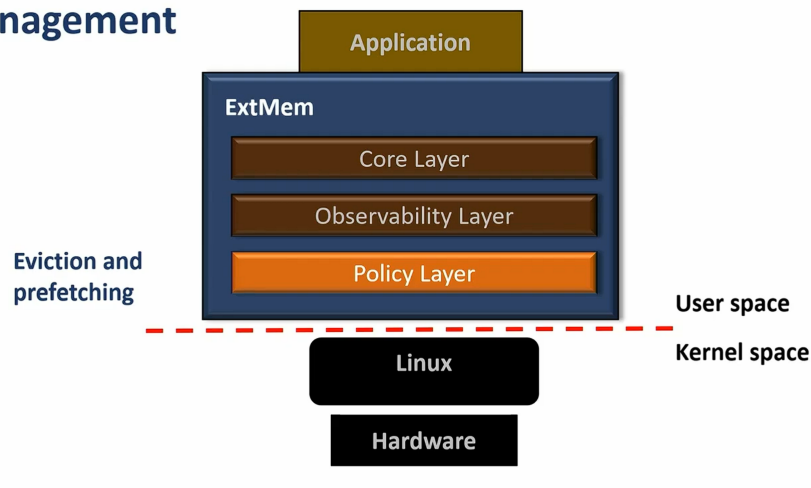


ExtMem: a framework for application-specific memory management



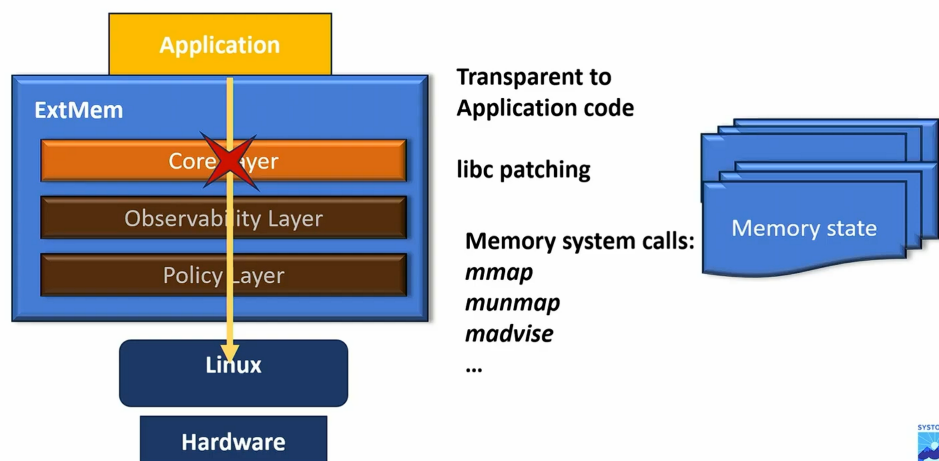
Observability Layer: 负责收集用户自定义策略所需的数据

ExtMem: a framework for application-specific memory management



Policy Layer: 向用户态开放API, 以实现替换、预取等内存管理策略

Core layer



core layer:

Extmem为了让用户能在用户态实现自定义策略, 必须将page fault事件从内核态传递到用户态

这一过程需要使用userfaultfd机制：

当应用程序进行内存系统调用时（比如mmap、munmap、madvise），core layer拦截这些调用，将内存区域注册到userfaultfd

当应用程序发生page fault时，若uffd检测出错误来自之前在mmap时，core layer注册到uffd的内存区域，则将错误转发给EXTMEM处理，再把内存管理决策权交给用户态

(syscall_interupt、libc patching——LD_PRELOAD)

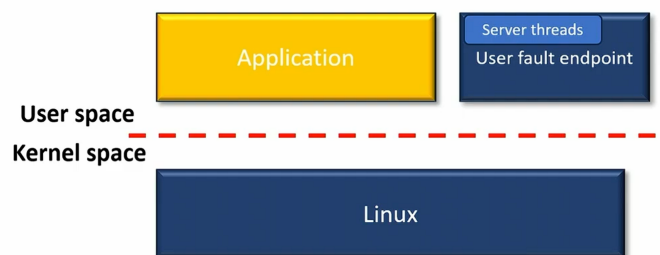
(mmap——在调用进程的虚拟地址空间中创建一个新的内存映射

munmap——删除指定地址范围内的内存映射

madvise——向内核提供关于一块内存区域未来使用模式的建议或提示)

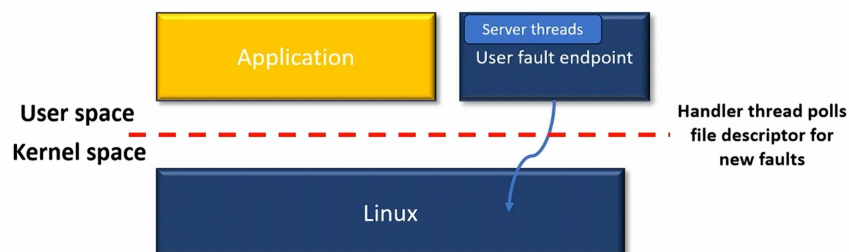
State of the art doesn't scale

Prior work based on userfaultfd doesn't scale due to expensive IPC and serialization



State of the art doesn't scale

Prior work based on userfaultfd doesn't scale due to expensive IPC and serialization



原本的userfaultfd机制是这样的：

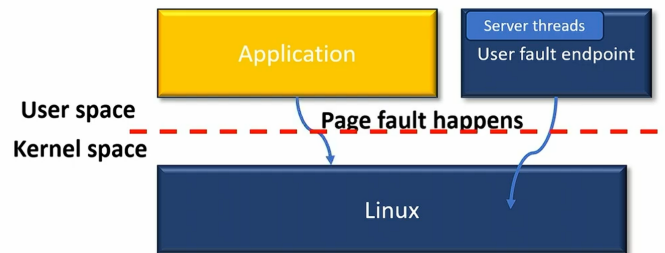
首先，在应用程序调用mmap操作的时候，core layer将其拦截下来，将为其分配的内存区域注册到userfaultfd

(Core Layer 调用 `userfaultfd()`，从内核获取一个代表 `userfaultfd` 服务本身的文件描述符 (`uffd`)，Core Layer 接着使用这个 `uffd`，通过 `ioctl` 命令，将刚刚 `mmap` 的那块内存区域注册到这个服务中去)

此时，user fault endpoint——也就是处理page fault的线程，（通过 `poll` 系统调用）监听 `userfaultfd` 的消息队列，等待新的page fault发生

State of the art doesn't scale

Prior work based on `userfaultfd` doesn't scale due to expensive IPC and serialization

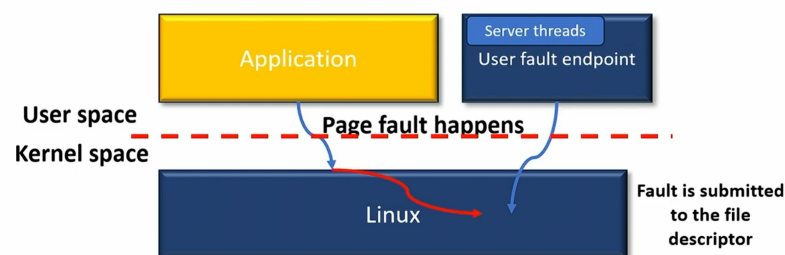


而应用程序之前的`mmap`操作，只是在进程的**虚拟地址空间**中预留了一段连续的地址范围，**但没有为它分配任何实际的物理内存**

当应用程序对这块`mmap`分配的虚拟地址空间进行读写操作时，由于没有映射到物理地址空间，会发生 page fault，应用程序由用户态转入内核态

State of the art doesn't scale

Prior work based on `userfaultfd` doesn't scale due to expensive IPC and serialization

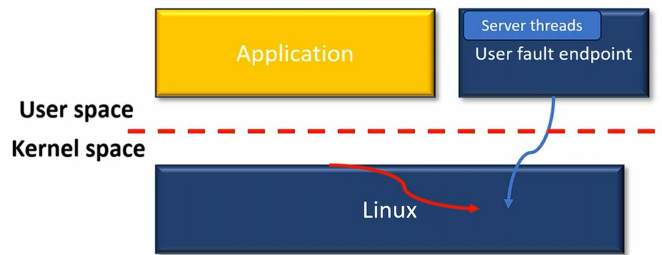


内核将这次缺页的关键信息，如**故障地址**、是**读缺页**还是**写缺页**打包成一个msg，送入`userfaultfd`的消息队列中

然后阻塞应用程序进程

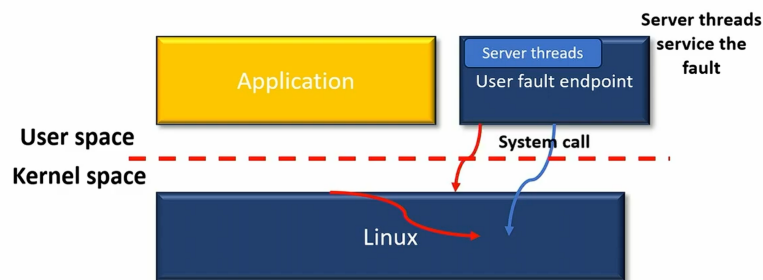
State of the art doesn't scale

Prior work based on userfaultfd doesn't scale due to expensive IPC and serialization



State of the art doesn't scale

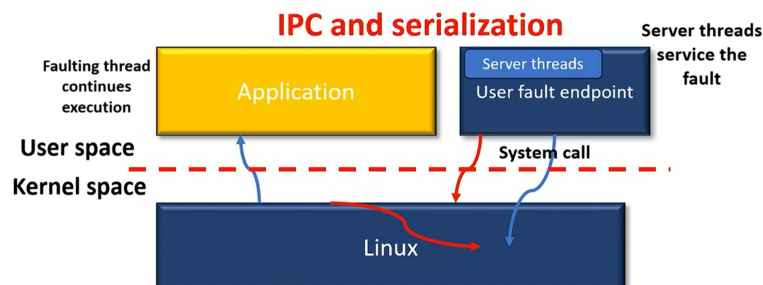
Prior work based on userfaultfd doesn't scale due to expensive IPC and serialization



user fault endpoint(Handler thread)从消息队列中取出msg, 对其进行处理

State of the art doesn't scale

Prior work based on userfaultfd doesn't scale due to expensive IPC and serialization

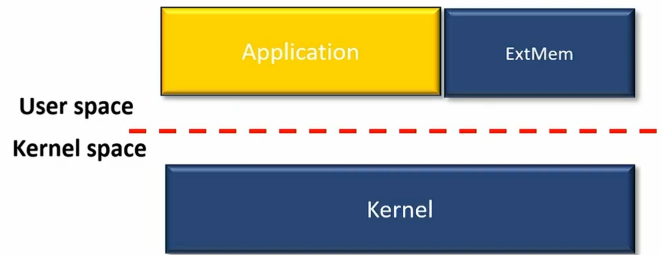


待其处理完毕, 唤醒应用程序进程继续运行

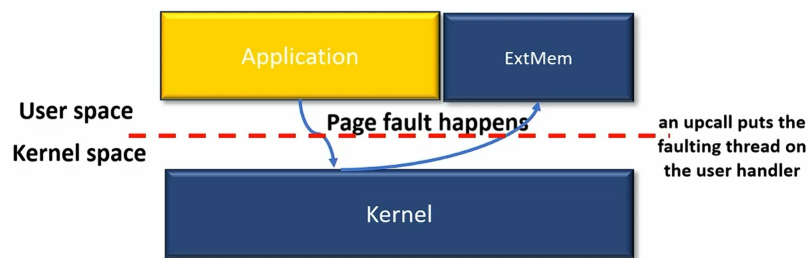
faulting thread和handler thread构成IPC机制

一个 handler 线程处理所有缺页, 在高并发下会成为瓶颈; 若采用多个handler, userfaultfd只有一个消息队列, 所有handler对其加锁读, 无法发挥handler并发优势

ExtMem scalable design



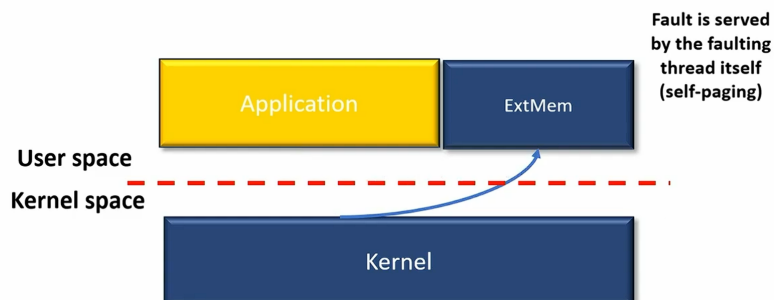
ExtMem scalable design



首先，在应用程序调用mmap操作的时候，core layer将其拦截下来，将为其分配的内存区域注册到userfaultfd，同时，为该进程注册一个upcall handler

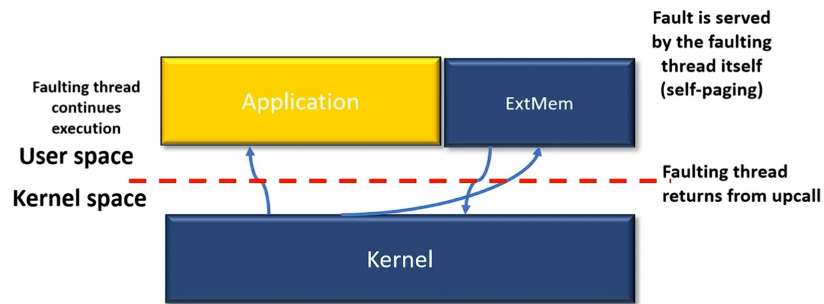
每当应用程序触发page fault，该fault都会转发到同一进程的用户态

ExtMem scalable design



由upcall handler进行处理

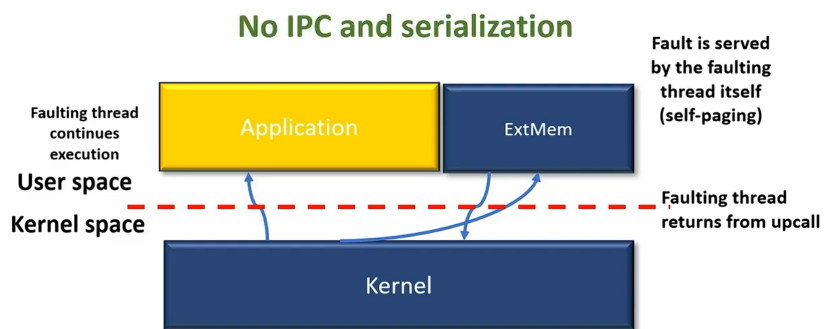
ExtMem scalable design



15

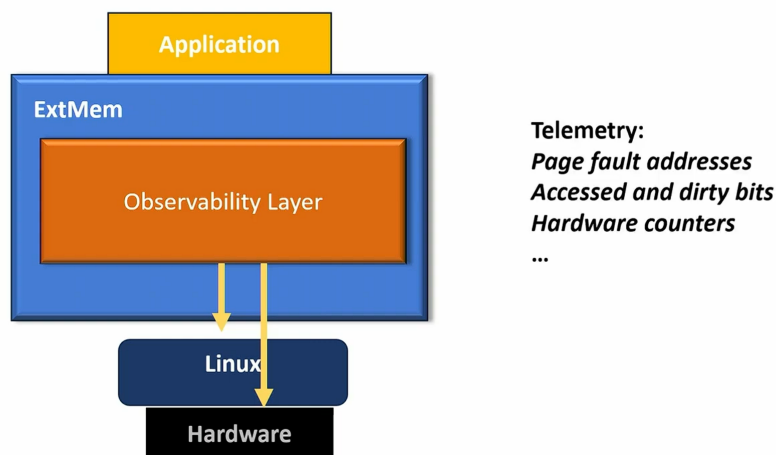
处理完返回应用程序继续执行

ExtMem scalable design



15

Observability layer

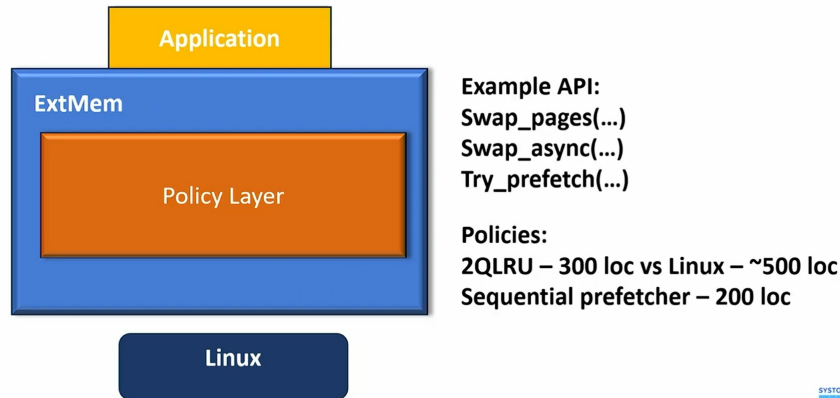


16

- page fault的故障地址
- MMU访问位和脏位
- Hardware Counter (硬件计数器)

- 位于 CPU 核心中的一组特殊寄存器。它们由一个称为 **PMU (Performance Monitoring Unit, 性能监控单元)** 的硬件子系统管理
- Observability Layer 会**定期地**读取 PMU 中这些计数器的值（执行了多少条指令、发生了多少次缓存未命中 (Cache Misses)、内存带宽的使用情况等）

Policy layer



Evaluation

Q1 Is ExtMem's upcall method better than userfaultfd?

Q2 Is the overhead of ExtMem reasonable?

Q3 How do ExtMem's policies compare to Linux?

Q4 Can we improve application performance using ExtMem?

System Setup:
 Intel Xeon 5218, 32 cores
 198 GB DDR4 DRAM
 NVMe 2.7 GB/s SSD

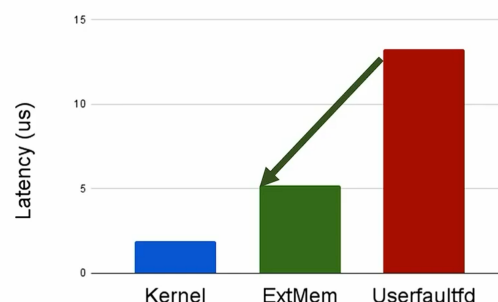


关于ExtMem的性能评估，列出了四个问题（这里主要说明第一个和第四个）

首先，是否ExtMem实现的upcall机制比userfaultfd要好？

Upcall minor page fault latency

ExtMem's upcall has better latency than userfaultfd by eliminating expensive IPC's

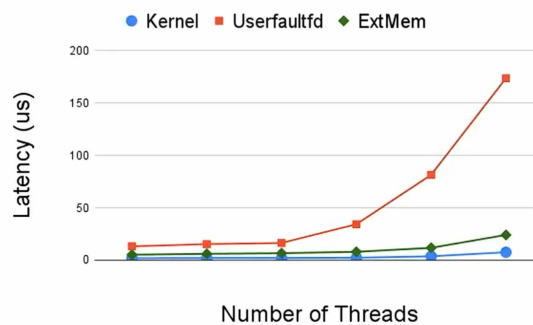


使用单个线程持续访问新分配的内存区域中的页面，记录解决单个page fault的平均延迟（当页面在内存中但未映射到进程页表中时，会出现minor page fault，衡量缺页处理机制本身的路径长短和开销）

由于usefaultfd故障处理路径中的往返IPC，ExtMem的延迟明显比userfaultfd的延迟低

Scalability with concurrent faulting threads

ExtMem's upcall has better scalability than userfaultfd by eliminating serialization

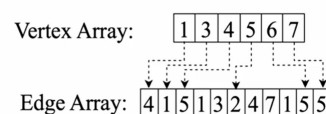
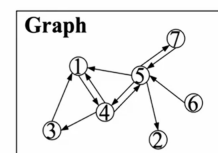


接着，使用不同数量的线程持续访问新分配的内存区域中的页面，记录在多线程并发环境下解决单个page fault的平均延迟

由于userfaultfd的待处理队列的读取需要上锁，不能进行并发的读取，发现随着线程数量的增多，ExtMem的延迟明显低于userfaultfd，且差距越来越大

Improving Page rank performance

- A large scale graph processing: PageRank
- Intuition: Vertex array stays hot and edge array is cold after being used once.
- Custom Policies:
 - Pin vertex array
 - Discard + prefetch window on edge array



第二个要说明的问题是，ExtMem是否能改善应用程序的性能？

论文运行了一个大规模的图处理程序PageRank

- 计算图中每个顶点重要性——反复迭代，根据邻居节点的值来更新每个节点自身的Rank值，直到所有节点的Rank值收敛稳定
- 当一个顶点被访问时，其相邻顶点档大概率被访问

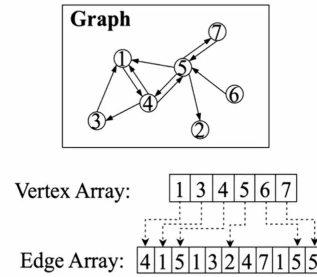
它由顶点数组和边数组两个数组组成

顶点数组：当前index顶点指向边数组中 该顶点自身的邻居顶点列表 的起始位置

边数组：按index存放每个顶点的邻居顶点列表

Page Rank System setup

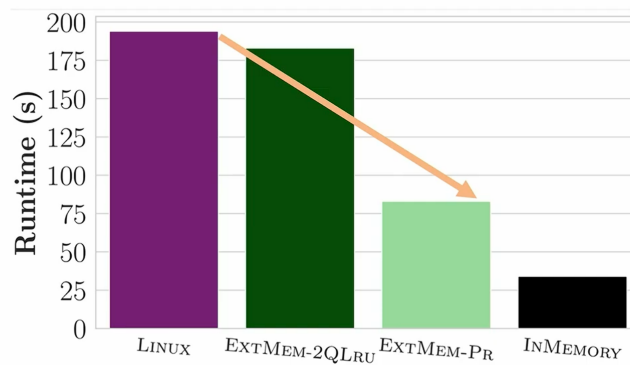
- Running GAP benchmark suite
- twitter dataset
- 50% (~7GB) fitting in memory
- 4 threads
- Runtime of 10 iterations measured



将内存限制在总图大小的50%

Case Study: Page Rank

ExtMem's special policy (ExtMem-PR) has better performance than the Linux default policies



使用与Linux (EXTMEM-2QLRU) 几乎相同的算法的EXTMEM会产生适度的改进

- 测试内存压力下的综合性能，除了上述缺页处理机制本身的路径长短和开销，还要考虑页面载入和驱逐问题：当需要的页面不在内存中，必须从磁盘上加载回来，当内存满了，要将页面从内存中驱逐到磁盘空间（handler的部分）
- 而应用程序专属的handler相比于linux为所有应用程序设计的通用handler更快
 - linux handler：不能因为一个程序需要内存，就粗暴地换出另一个重要程序的页面。它必须在所有进程之间做出权衡
(包含了大量的检查和复杂的逻辑分支：遍历全局的LRU、这个页面属于哪个进程？这个进程是不是已经换出了很多页？——为了公平，不能总从同一个“老实”进程里抢页面)
 - ExtMem：只需要考虑当前这一个应用程序的内存需求，不需要在多个进程间寻求公平
(从自己的LRU 链表的末尾取出一个页面)

但当我们部署自定义算法 (EXTMEM-PR) 时，我们获得了超过2倍的加速

EXTMEM-PR

2Q-LRU

